



Machine Learning PROJECT REPORT

Section:M11

Student Names:

Raed Abdulsalam Al-Maghdoowi : 4403569

Youssef Hluil Al-Juhani : 4402962

Majed Mohammed Al-Harbi : 4403241



Pages	Table of CONTENTS
1	Abstract and Introduction
2-3	Dataset and Domain Description
4-6	Theoretical Background
7-13	Methodology
14-22	Results and Evaluation
23-24	Discussion and Conclusion
25	References
26	Contribution report

Abstract and Introduction

A lot of people worldwide get rejected everyday because of nonexistent or limited credit histories, which unfortunately make them vulnerable to untrustworthy lenders.

Our dataset goal is to achieve financial inclusion by providing a safe and responsible lending experience for underserved populations. In order to make sure this happens, our dataset collects extensive alternative data to predict the client's ability to repay loans.

We aim to implement a machine learning model that will accurately predict who will repay and who will not, which will ensure that any trustworthy client will not be rejected. Finally, our goals in this study are:

- First, to preprocess the high-dimensional dataset.
- Second is to develop multiple supervised learning models that can classify and predict the clients who will default and who will not, such as Decision Tree, Logistic Regression, and Random Forest.
- Third to evaluate models performance using precision, recall, F1-score, AUC and other metrics.

Dataset and Domain Description

- Our dataset is Home Credit Default Risk from [Kaggle.com](https://www.kaggle.com),
- it is a financial dataset gathered to help lenders to observe the information of the clients that will default on a loan.
- It contains 122 features and 307,511 samples.
- The features have various data types, like information about the client (age, employment duration, family status like married or single or separated, etc.) and finances (type of income, annuity, total income, etc.).
- Our dataset is not clean and has a lot of NaN values and needs to be preprocessed, so we dealt with missing values by filling the missing numeric values with median, and filling the missing category values with unknown , encoding binary features, and we normalized the variation in feature scales, also we feature engineering the needed features.
- **Domain Background:** It is for financial understanding of the repayment behavior of clients to provide safe lending. Clients with limited or no credit history have a high chance of being vulnerable to untrustworthy lenders. By using our dataset and our machine learning models, lenders can make accurate decisions, which will help to reduce default rates.

Potential Challenges

Our dataset has great challenges that must be identified before starting to implement any machine learning models.

- One of the main challenges is class imbalance, the clients who will default on a loan are fewer than those who repay. The imbalance will cause the models to predict the class majority, which will lead to very poor results of high-risk clients.
- Another challenge that we encounter in the dataset is missing information, for many features have incomplete or missing values, this will require very careful handling.
- The dataset contains many categorical features that needs to be converted to numeric features.
- Also, our dataset comes from a real-world sources, naturally it will be harder to handle than created dataset.

These challenges show how important the preprocessing is for accurate, consistent models.

Theoretical Background

First Model: (Decision Tree)

A decision tree is a non-parametric supervised learning algorithm, which is utilized for both classification and regression tasks. It has a hierarchical tree structure, which consists of a root node, branches, internal nodes and leaf nodes.

Decision tree learning employs a divide and conquer strategy by conducting a greedy search to identify the optimal split points within a tree. This process of splitting is then repeated in a top-down, recursive manner until all, or the majority of records have been classified under specific class labels.

To choose the best attribute for splitting at each node algorithm use information gain and Entropy.

Entropy:

Entropy is a measurement to the impurity of the sample values. Entropy values can fall between 0 and 1. If all samples in data set, S , belong to one class, then entropy will equal zero. If half of the samples are classified as one class and the other half are in another class, entropy will be at its highest at 1. Entropy is defined by the following formula:

$$\text{Entropy}(S) = - \sum_{c \in C} p(c) \log_2 p(c)$$

Where:

- S represents the data set that entropy is calculated.
- c represents the classes in set, S .
- $p(c)$ represents the proportion of data points that belong to class c to the number of total data points in set, S .

Theoretical Background

cont...

Information Gain :

Information Gain represents the difference in entropy before and after a split on a given attribute. The attribute with the highest information gain will produce the best split. Information gain is usually represented with the following formula:

$$\text{Information Gain}(S, a) = \text{Entropy}(S) - \sum_{v \in \text{Values}(a)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Where:

- (a) represents a specific attribute or class label.
- Entropy (S) is the entropy of dataset, S.
- $|S_v|/|S|$ represents the proportion of the values in S_v to the number of values in dataset, S.

Second Model: (Random Forest):

Random Forest is a machine learning algorithm that uses many decision trees to make better predictions. Each tree looks at different random parts of the data, and their results are combined by voting for classification or averaging for regression which makes it as ensemble learning technique. This helps in improving performance and handling large datasets with many features, just as in our dataset.

A key advantage of Random Forest over Decision Tree is better generalization performance. Since Decision trees tend to tightly fit all the samples within training data they run the risk of overfitting. However, when there's a robust number of Decision trees in a random forest, the classifier won't overfit the model since the voting of many trees lowers the overall variance and prediction error. And that is what we noticed when we applied both algorithms.

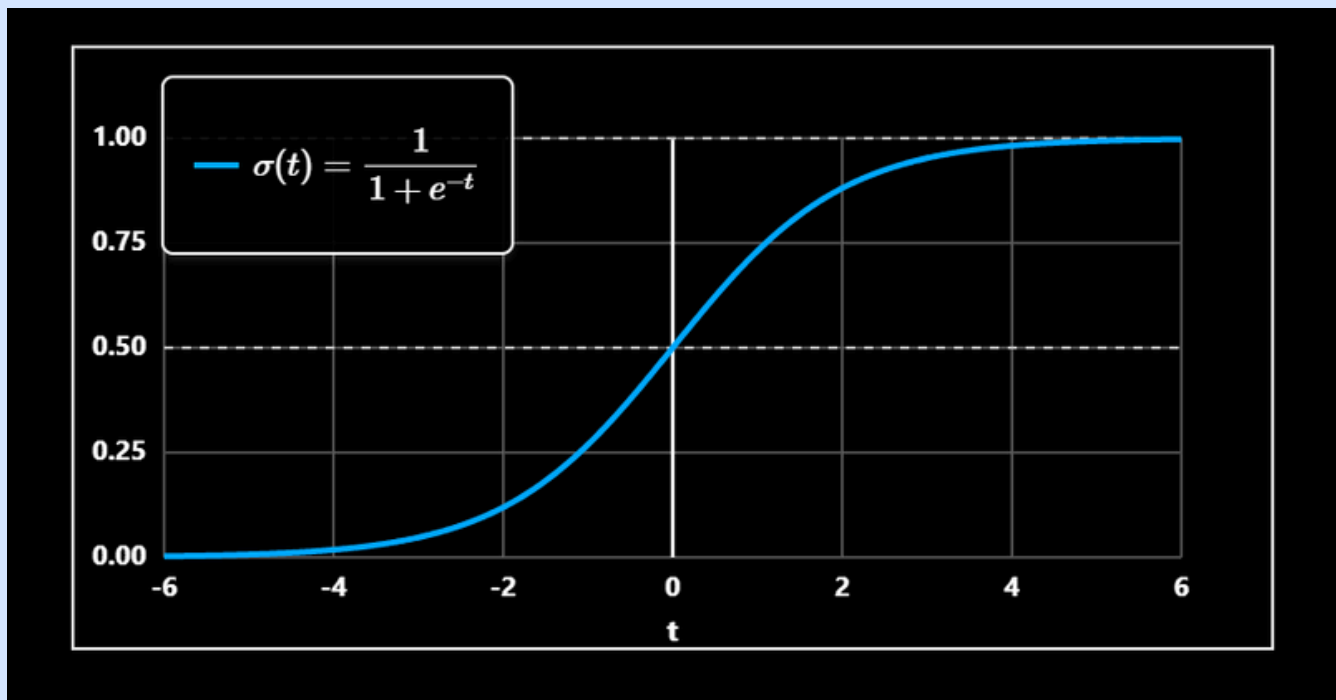
Theoretical Background

cont...

Third model: (Logistic Regression)

Logistic regression is commonly used to estimate the probability that an instance belongs to a particular class (for our dataset the probability that a customer will repay).

logistic regression work Just like a linear regression model, a logistic regression model computes a weighted sum of the input features (plus a bias term), but instead of outputting the result directly like the linear regression, it plugs the result into a sigmoid function that outputs a number between 0 and 1, If the estimated probability is greater than a given threshold (typically 50%), then the model predicts that the instance belongs to that class (called the positive class, labeled “1”), and otherwise it predicts that it does not (it belongs to the negative class, labeled “0”). This makes it a binary classifier, so it is suitable for our dataset which has binary target values. Here is an example of a sigmoid function:



Methodology

data preprocessing pipeline:

First, we need to split categorical and numerical columns to preprocess each type separately.

```
categorical_columns = X.select_dtypes(include="object").columns
numeric_columns = X.select_dtypes(exclude="object").columns
```

1- Handling missing values:

missing values in numeric features were filled using the median:

```
numeric_imputer = SimpleImputer(strategy='median')
X_train[numeric_columns] = numeric_imputer.fit_transform(X_train[numeric_columns])
X_test[numeric_columns] = numeric_imputer.transform(X_test[numeric_columns])
```

missing values in categorical features were filled with a new category "unknown".

```
categorical_imputer = SimpleImputer(strategy='constant', fill_value='Unknown')
X_train[categorical_columns] = categorical_imputer.fit_transform(X_train[categorical_columns])
X_test[categorical_columns] = categorical_imputer.transform(X_test[categorical_columns])
```

2- Feature Engineering For DAYS_BIRTH feature::

Before converting:

```
# In the dataset, we have age in days
df['DAYS_BIRTH'].head()
```



DAYS_BIRTH	
0	-9461
1	-16765
2	-19046
3	-19005
4	-19932

After converting:

```
# we convert the age to be in years
df['YEARS_AGE'] = ((-df['DAYS_BIRTH']/365).astype(int))

df['YEARS_AGE'].head()
```



YEARS_AGE	
0	25
1	45
2	52
3	52
4	54

Methodology

3- Normalization and Encoding:

We used a Column Transformer to apply Normalization and Encoding in one step.

```
scaler = StandardScaler()
encoder = OrdinalEncoder()

preprocessor = ColumnTransformer(
    [
        ("LabelEncoder", encoder, categorical_columns),
        ("StandardScaler", scaler, numeric_columns),
    ],
)
```

Column Transformers:

Allow different column subsets of the input data to be transformed separately and the features generated by each transformer will be concatenated to form a single feature space.

Parameters:

Transformers: list of tuples (name, transformer, columns) specifying the transformer objects to be applied to subsets of the data. This is the only parameter we used.

Applying the Column Transformer to the data:

```
X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)
```

Methodology

cont...

Data before and after preprocessing:

Data before preprocessing:

	SK_ID_CURR	TARGET	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL	AMT_CREDIT	AMT_ANNUITY	...
0	100002	1	Cash loans	M	N	Y	0	202500.000	406597.5	24700.5	...
1	100003	0	Cash loans	F	N	N	0	270000.000	1293502.5	35698.5	...
2	100004	0	Revolving loans	M	Y	Y	0	67500.000	135000.0	6750.0	...
3	100006	0	Cash loans	F	N	Y	0	135000.000	312682.5	29686.5	...
4	100007	0	Cash loans	M	N	Y	0	121500.000	513000.0	21865.5	...
5	100008	0	Cash loans	M	N	Y	0	99000.000	490495.5	27517.5	...
6	100009	0	Cash loans	F	Y	Y	1	171000.000	1560726.0	41301.0	...
7	100010	0	Cash loans	M	Y	Y	0	360000.000	1530000.0	42075.0	...
8	100011	0	Cash loans	F	N	Y	0	112500.000	1019610.0	33826.5	...
9	100012	0	Revolving loans	M	N	Y	0	135000.000	405000.0	20250.0	...
10	100014	0	Cash loans	F	N	Y	1	112500.000	652500.0	21177.0	...
11	100015	0	Cash loans	F	N	Y	0	38419.155	148365.0	10678.5	...
12	100016	0	Cash loans	F	N	Y	0	67500.000	80865.0	5881.5	...
13	100017	0	Cash loans	M	Y	N	1	225000.000	918468.0	28966.5	...
14	100018	0	Cash loans	F	N	Y	0	189000.000	773680.5	32778.0	...

Data after preprocessing:

	0	1	2	3	4	5	6	7	8	9	...	112	113	114	115	116	117	118	119	120	121
0	0.0	1.0	0.0	0.0	6.0	1.0	1.0	1.0	1.0	2.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	0.901626	-0.399034	0.695303	-1.451694
1	0.0	0.0	0.0	1.0	6.0	1.0	4.0	0.0	1.0	6.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	-0.435083	1.050252
2	0.0	0.0	0.0	0.0	6.0	3.0	4.0	1.0	3.0	17.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	0.695303	1.717437
3	0.0	0.0	1.0	1.0	1.0	7.0	4.0	1.0	1.0	8.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	-0.435083	-0.450916
4	0.0	0.0	0.0	1.0	6.0	7.0	4.0	0.0	1.0	14.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	-0.435083	0.216270
5	0.0	0.0	0.0	0.0	6.0	3.0	4.0	1.0	1.0	17.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	0.901626	-0.399034	-0.435083	1.217048
6	0.0	0.0	0.0	0.0	6.0	4.0	4.0	1.0	5.0	11.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	0.130110	-0.534314
7	0.0	0.0	0.0	1.0	6.0	3.0	4.0	1.0	1.0	17.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	0.130110	1.133650
8	0.0	1.0	1.0	1.0	6.0	7.0	4.0	1.0	1.0	4.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	0.901626	-0.399034	0.695303	0.966854
9	0.0	1.0	1.0	1.0	1.0	7.0	1.0	1.0	1.0	10.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	0.130110	-0.033925
10	0.0	1.0	1.0	0.0	6.0	7.0	4.0	1.0	1.0	4.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	-1.000276	-1.034703
11	0.0	0.0	1.0	1.0	6.0	7.0	1.0	0.0	1.0	3.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	0.901626	-0.399034	-0.435083	-1.785286
12	0.0	0.0	0.0	1.0	1.0	7.0	1.0	1.0	1.0	8.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	0.130110	-0.117323
13	0.0	0.0	1.0	1.0	6.0	7.0	1.0	3.0	1.0	14.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	-1.000276	-0.200721
14	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	1.0	8.0	...	-0.025677	-0.019515	-0.016236	-0.073262	-0.062499	-0.153041	-0.270030	-0.399034	-0.435083	-0.701110

Methodology

cont...

Train Models:

We implemented three supervised learning algorithms which are: Decision Tree – Random Forest – Logistic Regression. Our strategy is to implement these algorithms with almost default configurations and then apply Hyperparameters Tuning for the most promising model. This will help minimizing required resources and running time of the code. To handle the imbalance in the target class we will test each model twice, once with their default parameters and once with `class_weight = 'balanced'`. This will use the values of `y` to automatically adjust weights inversely proportional to class frequencies in the input data.

- **Decision Tree:**

```
model = DecisionTreeClassifier(random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[: ,1]
```

- **Random Forest:**

```
rf_model = RandomForestClassifier(
    n_estimators=500,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)
y_prob_rf = rf_model.predict_proba(X_test)[: ,1]
```

'n_estimator = 500' build 500 tree each one look at a random subset of the dataset. We chose 500 to enable the model to capture complex patterns that we think our dataset involve.

n_jobs = -1 means the number of CPU cores to use, -1 means to use every core in your computer which is the fastest option.

Methodology

cont...

Train Models:

- **Logistic Regression:**

```
LR_model = LogisticRegression( random_state=42, n_jobs=-1)

LR_model.fit(X_train, y_train)

y_pred_LR = LR_model.predict(X_test)
y_prob_LR = LR_model.predict_proba(X_test)[: ,1]
```

Evaluation:

we have imbalance in our dataset (8% for class 1 and %92 for class 0). In this case accuracy is not a good metric because if the model predicts that all samples belong to class 0 the accuracy will be %92 which is obviously a wrong result. models will tend to ignore minority class if we use accuracy as our main metric.

Also, in our problem false negatives have high cost since the problem is about detecting customers that will not repay the loan. In such cases Recall is the most appropriate metric. Recall measures how often a machine learning model correctly identifies positive instances (true positives) from all the actual positive samples in the dataset.

- **Recall :**

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

For these reasons, Recall will be our primary metric. We will also use other metrics, including Precision, F1-score and Area under the ROC curve (AUC).

Methodology

cont...

- **Precision:**

Precision shows how often an ML model is correct when predicting the target class. we can calculate precision by dividing the number of correct positive predictions (true positives) by the total number of instances the model predicted as positive (both true and false positives). Precision is useful when the cost of a false positive is high.

$$\text{Precision} = \frac{\text{correctly classified actual positives}}{\text{everything classified as positive}}$$

- **F1-score:**

The F1 score is the harmonic mean of precision and recall.

$$F1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{2TP}{2TP + FP + FN}$$

- **Area under the ROC curve (AUC)**

Receiver-operating characteristic curve (ROC): The ROC curve is a visual representation of model performance across all thresholds. The ROC curve is drawn by calculating the true positive rate (TPR) and false positive rate (FPR) at every possible threshold, then graphing TPR over FPR.

Area under the ROC curve (AUC): The area under the ROC curve (AUC) represents the probability that the model, if given a randomly chosen positive and negative example, will rank the positive higher than the negative.

$$\begin{array}{c} \text{Recall} \\ \text{Sensitivity} \\ \text{True positive rate (TPR)} \end{array} \frac{TP}{FN + TP} = \frac{TP}{P} \quad \begin{array}{c} \text{False positive rate (FPR)} \\ \text{False alarm rate} \end{array} \frac{FP}{TN + FP} = \frac{FP}{N}$$

Methodology

cont...

Train Models:

Hypermeter Tuning for the most promising model
according to Recall metric: (Logistic Regression)

```
✓ LR_params = {  
    'C' : [100, 10, 1.0, 0.1, 0.01],  
    'solver' : ['newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'],  
}  
  
✓ random = RandomizedSearchCV(estimator=LogisticRegression(random_state=42, class_weight='balanced')  
                               param_distributions=LR_params,  
                               n_iter=20,  
                               cv=3,  
                               verbose=0,  
                               random_state=42,  
                               n_jobs=-1,  
                               scoring = 'recall')  
  
print(f"----- Best Params -----")  
random.fit(X_train, y_train)  
print(random.best_params_)
```

- C: Inverse of regularization strength
- solver: The optimization algorithm used to train Logistic Regression.

Results and Evaluation

before the class balance

First Model: (Decision Tree)

Classification Report:

Here are the classification report and AUC before the class balance:

...	precision	recall	f1-score	support
0	0.92	0.91	0.92	34960
1	0.14	0.16	0.15	3089
accuracy			0.85	38049
macro avg	0.53	0.54	0.53	38049
weighted avg	0.86	0.85	0.85	38049
AUC: 0.5364275214776283				

Model accuracy: 85%.

Class (0):

Precision (0.92): High. If the model predicts a value of 0, the probability of it being correct is 0.92.

Recall (0.91): High. The model predicted 91% of all class 0s. It made approximately 9% of its prediction errors, resulting in class 1s.

F1 Score (0.92): High. The model's performance on class (0) is good and balanced between precision and recall.

Class (1):

Precision (0.14): Very Low. If the model predicts a value of 1, the probability of it being correct is 14%. This means there are a lot of false positives.

Recall (0.16): Very Low. The model only finds 16% of the actual Class 1 cases. It misses 84% of them.

F1 score (0.15): Very Low. This indicates a poor balance on class (1) between precision and recall.

Results and Evaluation

before the class balance

Second Model: (Random Forest)

Classification Report:

Here are the classification report and AUC before the class balance:

...	precision	recall	f1-score	support
0	0.92	1.00	0.96	34960
1	1.00	0.00	0.00	3089
accuracy			0.92	38049
macro avg	0.96	0.50	0.48	38049
weighted avg	0.93	0.92	0.88	38049
AUC: 0.7183080946045353				

Model accuracy: 92%.

Class (0):

Precision (0.92): High. If the model predicts a value of 0, the probability of it being correct is 0.92.

Recall (1.00): Perfect. The model predicted 100% of all class 0s. It made approximately 0% of its prediction errors, resulting in class 1s.

F1 Score (0.96): Very High. The model's performance on class (0) is good and balanced between precision and recall.

Class (1):

Precision (1.00): Very High. If the model predicts a value of 1, the probability of it being correct is 100%. This means there are no false positives.

Recall (0.00): Very Bad. The model only finds 0% of the actual Class 1 cases. It misses 100% of them.

F1 score (0.00): Very Bad. This indicates a poor balance on class (1) between precision and recall.

Results and Evaluation

before the class balance

Third model: (Logistic Regression)

Classification Report:

Here are the classification report and AUC before the class balance:

...	precision	recall	f1-score	support
0	0.92	1.00	0.96	34960
1	0.48	0.01	0.02	3089
accuracy			0.92	38049
macro avg	0.70	0.50	0.49	38049
weighted avg	0.88	0.92	0.88	38049
AUC: 0.7363870784573295				

Model accuracy: 92%.

Class (0):

Precision (0.92): High. If the model predicts a value of 0, the probability of it being correct is 0.92.

Recall (1.00): Perfect. The model predicted all class 0s. It made approximately no prediction errors, resulting in class 1s.

F1 Score (0.96): Very High. The model's performance on class (0) is good and balanced between precision and recall.

Class (1):

Precision (0.48): Moderate. In the very rare cases where the Class 1 model predicted, the correctness was 48%. However, because the recall value is 0.01, these predictions are not statistically significant (too few to be considered).

Recall (0.01): Very bad. This means the model missed approximately 99.99% of the actual Class 1 cases.

F1 score (0.26): Low. This low score reflects a poor balance between precision and recall.

Results and Evaluation

after the class (balance)

First Model: (Decision Tree)

Classification Report:

Model accuracy: 86%.

Class (0):

	precision	recall	f1-score	support
0	0.92	0.92	0.92	56538
1	0.14	0.15	0.15	4965
accuracy			0.86	61503
macro avg	0.53	0.54	0.53	61503
weighted avg	0.86	0.86	0.86	61503

Precision (0.92): High. If the model predicts a value of 0, the probability of it being correct is 0.92.

Recall (0.92): High. The model predicted 92% of all class 0s. It made approximately 8% of its prediction errors, resulting in class 1s.

F1 Score (0.92): High. The model's performance on class (0) is good and balanced between precision and recall.

Class (1):

Precision (0.14): Very Low. If the model predicts a value of 1, the probability of it being correct is 14%. This means there are a lot of false positives.

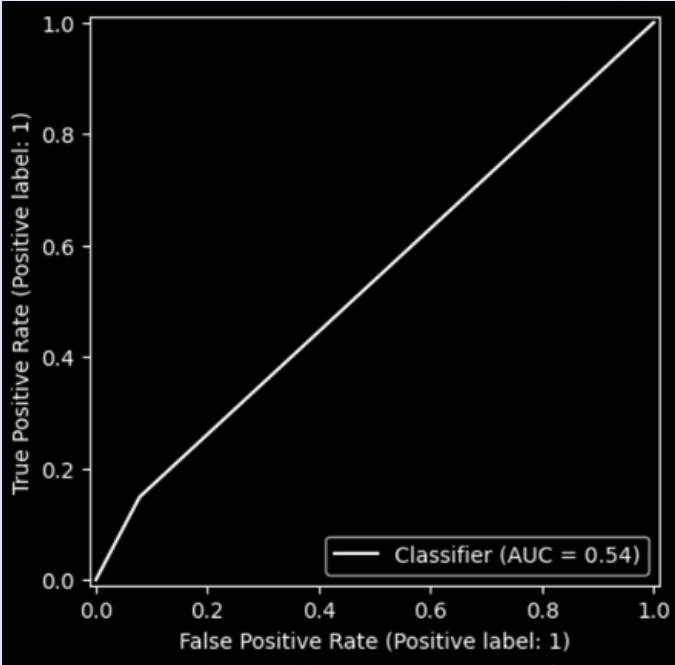
Recall (0.15): Very Low. The model only finds 15% of the actual Class 1 cases. It misses 85% of them

F1 score (0.15): Very Low. This indicates a poor balance between precision and recall.

ROC (Receiver Operating Characteristic)

The AUC value is 0.54, indicating random guess performance.

This means there is a 54% probability that the model will correctly distinguish between positive and negative cases.



Results and Evaluation

after the class (balance)

Second Model: (Random Forest)

Classification Report:

Model accuracy: 92%.

Class (0):

	precision	recall	f1-score	support
0	0.92	1.00	0.96	56538
1	0.46	0.00	0.00	4965
accuracy			0.92	61503
macro avg	0.69	0.50	0.48	61503
weighted avg	0.88	0.92	0.88	61503

Precision (0.92): High. If the model predicts a value of 0, the probability of it being correct is 0.92.

Recall (1.00): Perfect. The model predicted all class 0s. It made approximately no prediction errors, resulting in class 1s.

F1 Score (0.96): Very High. The model's performance is strong; this is primarily due to the strength of its Precision and Recall capabilities.

Class (1):

Precision (0.46): Moderate. In the very rare cases where the Class 1 model predicted, the correctness was 46%. However, because the recall value is 0.00, these predictions are not statistically significant (too few to be considered).

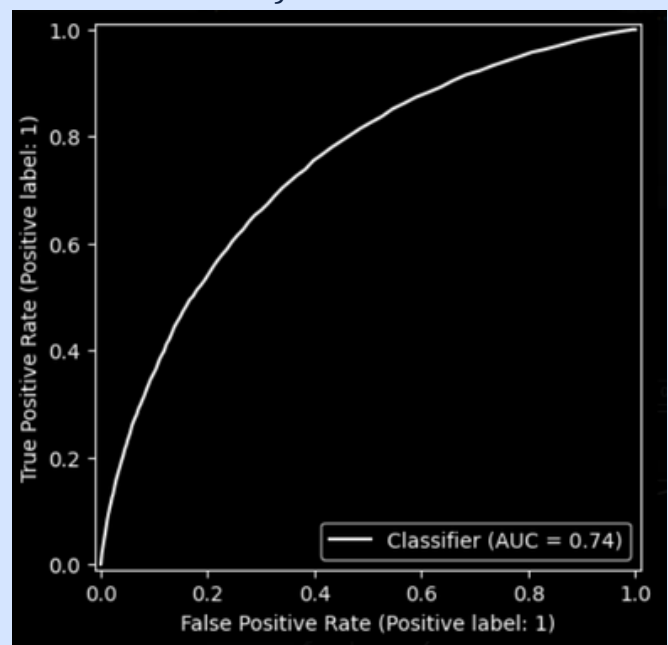
Recall (0.00): Very bad. This means the model missed approximately 100% of the actual Class 1 cases. It failed to identify any of the approximately 4965 cases.

F1-Score (0.00): bad. Since Recall is zero, the F1-score is also effectively zero.

ROC (Receiver Operating Characteristic):

The AUC value is 0.74, indicating good performance.

This means there is a 74% probability that the model will correctly distinguish between positive and negative cases.



Results and Evaluation

after the class (balance)

Third model: (Logistic Regression)

Classification Report:

Model accuracy: 69%.

Class (0):

	precision	recall	f1-score	support
0	0.96	0.69	0.80	56538
1	0.16	0.67	0.26	4965
accuracy			0.69	61503
macro avg	0.56	0.68	0.53	61503
weighted avg	0.89	0.69	0.76	61503

Precision (0.96): Very high. If the model predicts a value of 0, the probability of it being correct is 0.96.

Recall (0.69): Moderate. The model predicted only 69% of all class 0s. It made approximately 31% of its prediction errors, resulting in class 1s.

F1-Score (0.80): Good. The model's performance is good for the majority of classes.

Class (1):

Precision (0.16): Very Low. If the model predicts a value of 1, the probability of it being correct is 16%. This means there are a lot of false positives.

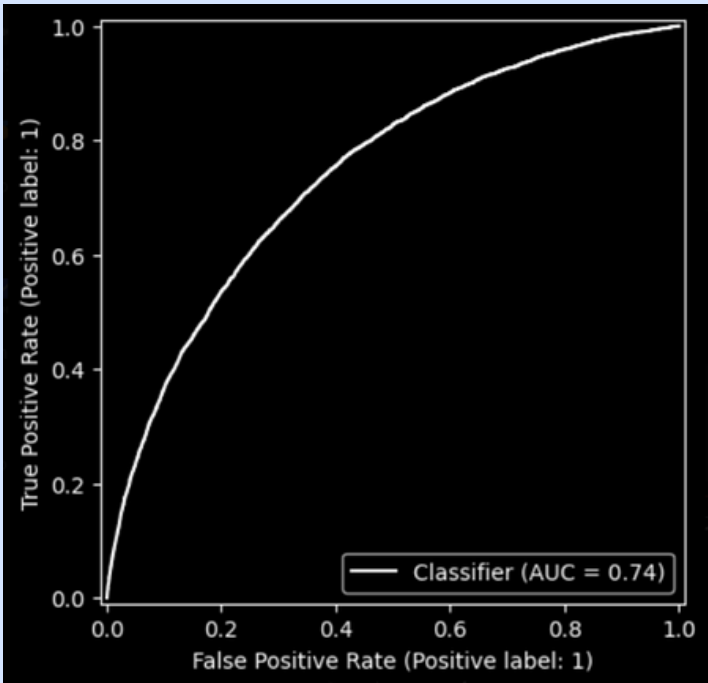
Recall (0.67): Moderate. Although the precision was low, the model successfully found 67% of the values in Class 1.

F1 score (0.26): Very Low. This low score reflects a poor balance between precision and recall.

ROC (Receiver Operating Characteristic):

The AUC value is 0.74, indicating good performance.

This means there is a 74% probability that the model will correctly distinguish between positive and negative cases.



Results and Evaluation

cont...

comparison after using class_weight balance:

To address the effect of class imbalance and in order to solve such a problem, we trained each model twice, first with no balancing whatsoever and then using the `class_weight = 'balance'`.

The overall performance for The Decision Tree model:

The accuracy almost remains unchanged, increasing only from 0.85 to 0.86. In the minority class before balancing, the model achieved a recall of 0.16, and after applying the class weighting, it is slightly decreased to 0.15. Precision and F1-score also showed no meaningful change in both classes the minority and majority. The AUC remained essentially the same (0.536 -> 0.534). We conclude that class weighting is not sufficient for improving The Decision Tree performance.

The overall performance for The Random Forest model:

The accuracy remained identical at 0.92 in both models. In the minority class before balancing, the model achieved a recall of 0.00, and after applying class weighting, it remained 0.00. Precision for the minority class changed from 1.00 to 0.50, F1-score did not change and remained 0.00. The majority class did not change at all. The AUC showed a small improvement, increasing from 0.718 to 0.734.

We conclude that class weighting did not improve the data imbalance in the Random Forest model because the recall and F1-score remained 0.00.

The overall performance for The Logistic Regression model:

The accuracy dropped from 0.92 to 0.69. In the minority class the recall showed a great improvement from 0.01 to 0.66. The Precision dropped from 0.48 to 0.16, and the F1-score improved from 0.02 to 0.26, for the majority class the recall dropped from 1.00 to 0.69, The Precision improved from 0.92 to 0.96 and the F1-score dropped from 0.96 to 0.80.

We can see that the class weighting had a good effect in balancing the data, and we can observe from the classification report that the data has become more balanced than the two previous models.

Results and Evaluation

cont...

Compare models and discuss performance differences:

The Decision Tree model:

performance was very poor even though the overall accuracy was pretty high (86%), because of the imbalanced dataset the model will predict the class (0) all the time and still achieve a very high accuracy, so the accuracy is not a good metrics here, This is observed by the poor precision, recall, and F1-score for the class (1).

The ROC-AUC is 0,54 which is says that the model is slightly better than random guess. The model is not reliable in distinguish the two classes.

Finally, the Decision Tree model shows that it suffers from overfitting and shows a poor handling imbalance data effectively.

The Random Forest model:

The Random Forest model achieved a moderate performance, the accuracy is the highest among the three model but if we observe the classification report we can see a great precision, recall, f1-score for the class 0 which means the model is extremely good when it comes to the majority class, but in the minority class the model completely fails, for the precision only 46% of predicted class 1 labels are correct which is good, but with a recall and f1-score equals 0.00 it failed to detect any actual class 1 samples. The weighted average shows how class 0 dominated due to dataset imbalance, that's why F1 weighted = 0.88 even it is 0.00 for class 1. The AUC shows how well the model separated class overall in term of probability space, it does not look at the predicted labels (0 or 1), it looks at their predicted probabilities this is why even though the model fails to predict class 1 it still assigns different probability values to class 1 vs class 0, with AUC = 0.74 it shows that the model can defernite between risky and non-risky customers but it fails to predict class 1 (recall = 0)

Results and Evaluation

cont...

Compare models and discuss performance differences:

The Logistic Regression model:

achieved the strongest performance among the three models, it produced a good accuracy and demonstrated superior precision, recall, and F1-score, which indicates that Logistic Regression was very effective when it comes to imbalanced data. When we see the classification report we can observe that it is much more balanced than Random Forest, The model shows great precision and a very good recall and F1-score for the majority class. For the minority class we can see that the precision dropped to 0.16 which is normal because the recall increased to 0.67 from 0.00 it is a huge improvement. The F1-score is 0.26 which is low due to a low precision the model is catching many class 1 samples but also mispredicting many class 0 as class 1, AUC = 0.7447 which is slightly better than Random forest, so the model still separates classes reasonably well. Finally the model now detects a lot of minority, Recall for class 1 went from 0 to 0.67, more balanced and better performance for finding class 1 samples.

The Optimized Logistic Regression model:

because the Logistic Regression model gave the best performance among the three model we decided to optimize the model through Hypermeter Tuning, sadly after applying the hypermeter tuning the model gave a slightly better performance, for the classification report it is the exact same result, but for the AUC it is better with 0.7469 instead of 0.7447 this is the only difference between the two model, which is says that this is the best performance that the model can do.

	precision	recall	f1-score	support
0	0.96	0.69	0.80	56538
1	0.16	0.67	0.26	4965
accuracy			0.69	61503
macro avg	0.56	0.68	0.53	61503
weighted avg	0.90	0.69	0.76	61503
AUC: 0.7468888074528706				

Discussion and Conclusion

Interpret the findings in the context of the domain:

The analysis of the three models-Decision Tree, Random Forest, and Logistic Regression shows a significant challenge: the dataset is highly imbalanced. this means most of the customers belong to class 0, while a small number belong to class 1. Class 0 means the customer who will pay back the loan, while class 1 means the customers will not pay.

The Random Forest model shows a high accuracy (92%), but in reality this is misleading due to the class imbalance. It acts like a bank that approves loaners without checking risks. Because most customers in the dataset pay back their loans, the model looks accurate. it fails to find the risky customers which is the whole point of this model.

The Decision Tree model also shows a poor performance, with an AUC of 0.54 it is like a bank who would randomly approves customers, means that they make decisions randomly, which is not suitable for financial companies that must manage risk carefully.

The Logistic Regression model performs best in a real-world circumstance, despite the lower overall accuracy (69%), it successfully identifies 67% of customers who are likely to default, it is like a bank that may reject some good customers, but it carefully protect itself from big losses by detecting high-risk customers early.

Implications for Decision-Making:

In risk prediction, the cost of a false negative (accepting a high-risk customer) is typically much higher than the cost of a false positive (rejecting a non-risk customer).

While the random forest model is highly accurate, it poses a risk to decision-making in this scenario because it ignores all potential risks.

The logistic regression model is the best decision-making model. Although its lower precision (0.16) means a higher false positive rate, it provides the necessary sensitivity to detect defaults or other potential risks.

In decision-making and specifically in financial, the cost of missing a risky customer is much worse than rejecting a safe one. if a bank approves a high-risk customer who will not repay, the bank can lose a huge amount of money. On the other hand, rejecting a loan from a low-risk customer only means potential profit lose.

Even though Logistic Regression model shows a low precision, this means it is more cautious than needs to. It may reject more customers, but this will reduces the chance of approving a high-risk loans. In real perspective, this is safer and more responsible decision making.

Therefore, in financial perspective, Logistic Regression is the most practical model, as it balances risk and security rather than focusing on high accuracy numbers.

Discussion and Conclusion

cont...

Potential improvements or extensions:

The results show that the model's performance is severely limited by data imbalance (where the number of class 0 significantly exceeds that of class 1). To improve future iterations of this project, the following strategies are recommended:

Resampling the data: Artificially rebalance the training data. This is likely to improve Precision and F1-scores without sacrificing the same level of accuracy.

Conclusion:

In conclusion, this study demonstrates that accuracy is not a reliable measure for imbalanced datasets. While logistic regression is the best-performing model, with an AUC of 0.747 and a recall ratio of 0.67 for Class 1, there is significant room for improvement in terms of precision. The key takeaway is that a model capable of identifying risks is far more valuable than one limited to statistical accuracy in safe scenarios.

Future work should prioritize dataset balancing to develop a robust and accurate risk identification tool.

Thanks for your time...

References

- **Book: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**
- **IBM website:**
<https://www.ibm.com/think/machine-learning#605511093>
- **GeeksforGeeks website:**
<https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>
-
- **Scikit learn documentation:**
<https://scikit-learn.org>
- **EVIDENTLY AI: classification metrics guide**
<https://www.evidentlyai.com/classification-metrics/accuracy-precision-recall#what-is-recall>
- **Google for Developers website:**
<https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>

Contribution report

Majed	Raed	Youssef
Data preprocessing.	Data preprocessing.	Data preprocessing.
Train Decision Tree.	Train Logistic Regression	Train Decision Tree.
Train Random Forest.	Hypermeter Tuning for the most promising model.	Abstract and Introduction.
Theoretical Background.	Results and Evaluation.	Dataset and Domain Description.
Methodology.	Discussion and Conclusion.	Results and Evaluation.